Resource Allocation System - Iteration 41

PostgreSQL Application Persistence

Najib Attar
October 2026

1 Iteration Goal

Replace operational CSV and local result-file persistence with PostgreSQL while preserving the C++17 allocation engine, JSON protocol v1 and existing Flask workflows. The database becomes the single operational source of truth; allocation decisions remain the responsibility of the existing C++ rules and strategies.

2 Changes from Iteration 40

- Introduce PostgreSQL persistence with SQLAlchemy, psycopg 3 and Alembic migrations.

- Store ordered inputs, allocation runs, decisions, allocations, suggestions and history relationally.

- Build JSON engine snapshots from the database and publish validated results transactionally.

- Use DatabaseResultAdapter for database-backed views without runtime file reads or fallback.

- Add safe CSV import, fictional demonstration data, integrity checks and recovery verification.

The JSON execution boundary was already available before the database migration. Iteration 41 changes persistence, not the engine protocol or allocation algorithms.

3 Scenario

An instructor saves a lecture, examination or committee request. Save Only commits the request and participants without allocation. Submit and Run saves first, captures an ordered snapshot, executes C++ and publishes one complete result. If execution fails, the new request remains saved and unprocessed; the last successful result remains visible. Dashboard, detail and schedule pages retrieve a consistent database view.

4 Modules and Their Tasks

Module	Responsibility
web/persistence/	Models, connection management, validation, repository transactions and explicit import.
allocation_workflow.py	Coordinate snapshot capture, engine execution and transactional result publication.
EngineGateway	Verify the executable and exchange protocol-v1 JSON using stdin/stdout.
ResourceAllocationSession	Coordinate existing C++ allocation services, rules, strategies and suggestions.
DatabaseResultAdapter	Reuse file-free result presentation for Flask views.
migrations/ and scripts/	Version schema changes; validate imports, seed demo data and generate ER documentation.

5 Database Tables and Relationships

The schema contains **18 application tables**. Alembic's `alembic_version` is a nineteenth physical table used only for migration tracking. The following inventory covers all current application tables; PK means primary key and FK means foreign key.

Table	Keys / Relationships	Purpose
role	PK id; unique name	Shared primary and assigned role labels.
organization_unit	PK id; FK parent_id	Named university units and optional hierarchy.
university_user	PK id; FKs primary_role_id, primary_unit_id	User identity and existing role-specific profile fields.
user_role	PK/FKs user_id, role_id	Ordered many-to-many assigned roles.
resource	PK id; unique source_order	Typed rooms, capacity, facilities, building and availability.
resource_request	PK id; FKs requester_id, resource_id	Ordered requests, requirements, revision and optional retry key.
request_slot	PK request_id, ordinal; FK request_id	Ordered weekly occurrences, including recurring requests.
exam_details	PK/FK request_id	Course metadata, exam type and room-splitting flag.
request_participant	PK/FKs request_id, user_id	Ordered required committee participants and role labels.
busy_slot	PK id; FK user_id	Independent weekly user commitments and reasons.
allocation_run	PK id; unique correlation_id	Captured inputs, strategy, revision, status and engine build.
application_state	PK id = 1; FK active_run_id	Selected strategy, input revision and active result run.
request_decision	PK/FKs run_id, request_id	Per-run status, rejection reason and immutable engine detail.
allocation	PK id; FKs (run_id, request_id), resource_id	Run-local engine identity, room, slot and assigned count.
allocation_participant	PK/FKs allocation_id, user_id	Required human commitments for an allocation.
suggestion_group	PK/FKs run_id, request_id	A suggestion scan, including a scan with no alternatives.
meeting_suggestion	PK run_id, request_id, rank; group/resource FKs	Ranked alternative room/time, score and explanation.
request_history	PK id; FKs request_id, optional run_id	Timestamped save, import and decision provenance.

Integrity: positive identifiers/capacities, valid weekday-minute intervals, unique ordering and participants, and composite result FKs prevent ambiguous or cross-run records. All FKs use NO ACTION rather than silently cascading deletion. Deferred subtype constraints require slots for every request, exam metadata only for exams and participant mappings only for committee requests.

Normalization: slots, roles, participants and suggestions use related tables. Immutable JSON snapshots and decision payloads intentionally retain historical engine detail. Current organization forests are validated before initialization and execution; direct SQL is not a supported alternative application write interface.

6 Updated Workflow Diagram

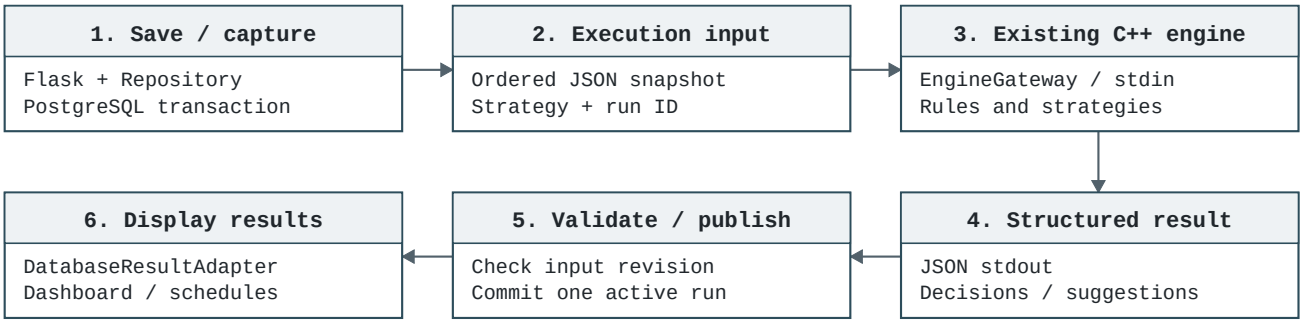

Figure 1. PostgreSQL owns persistence. C++ computation runs outside a database transaction; the returned structured result is validated before publication.

7 Updated ER Overview

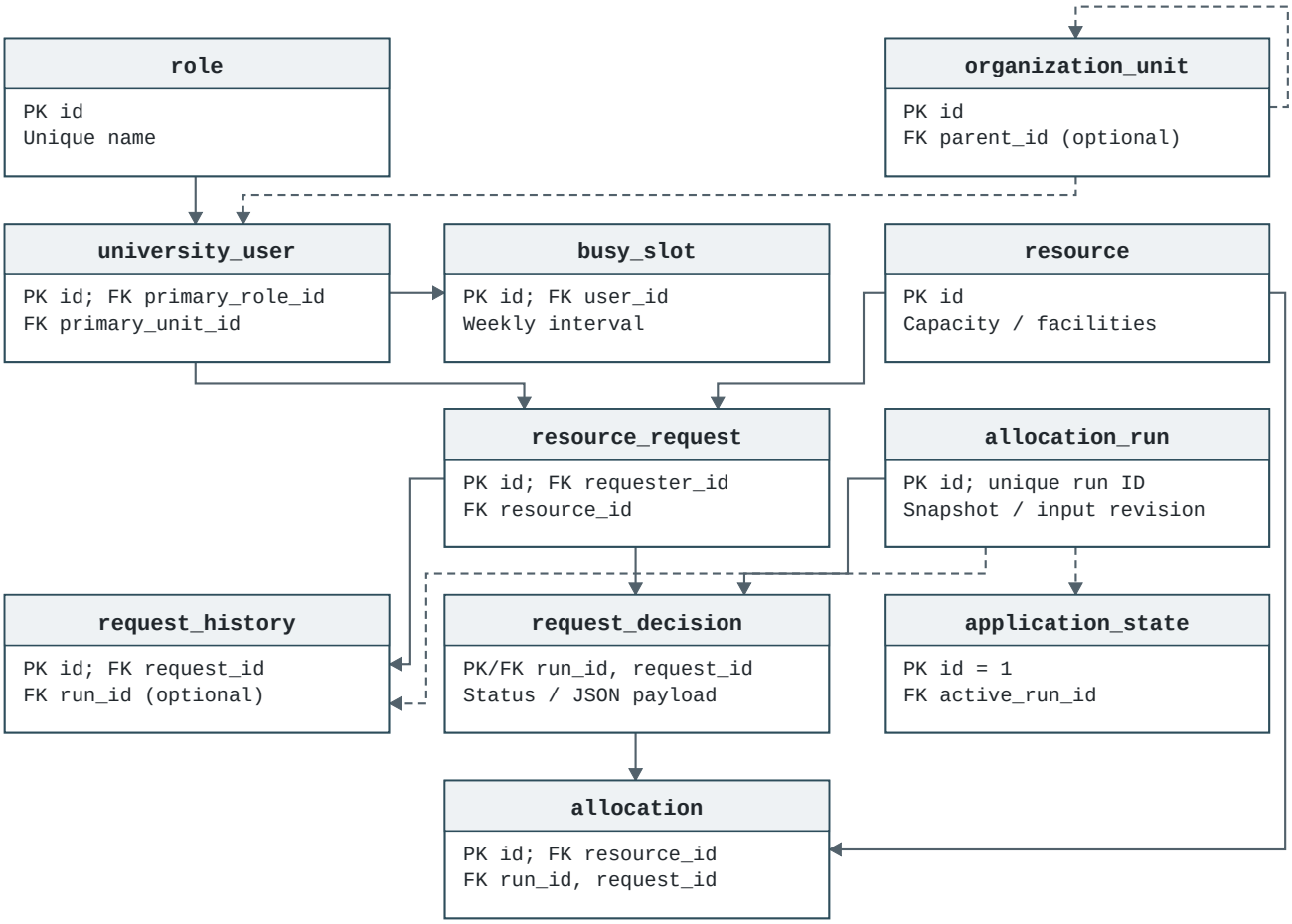

Figure 2. Principal FK relationships, from parent to referencing child. Dashed links are optional; most relationships are one-to-many. Supporting tables are listed on page 2.

Full columns, keys and cardinalities: docs/architecture/database-er-physical.svg, with its Mermaid source and schema catalog.

8 Prototype Implementation

The **Repository** separates SQL persistence from routes and allocation decisions. **DatabaseResultAdapter** reuses the existing Adapter-style presentation boundary. ResourceAllocationSession and RuleEngineFacade remain facades; existing strategies, factories and domain polymorphism are preserved. C++ has no PostgreSQL dependency.

Transactions and concurrency. Input writes briefly lock the singleton application state and increment its revision. PostgreSQL identities generate request IDs and submission order. A run records an ordered snapshot and releases the transaction before invoking C++. Publication locks the state again, checks the revision and run ordering, and atomically commits decisions, allocations, participants, suggestions, history and the active-run reference. Failed or stale runs never replace a successful publication. Readers use repeatable read.

Safe retries and errors. Exact keyed form retries reuse one saved request; conflicting content returns HTTP 409. Lock waits are bounded and contention returns HTTP 503. Database failures are explicit and do not trigger a file fallback. A crash releases database locks, although an interrupted engine run may retain a non-authoritative running record.

Allocation compatibility. All five strategies, minute-based recurring slots, multi-room exams, compatible shared-room exams and committee commitments remain engine responsibilities. No blanket SQL room-overlap constraint is added, because it would forbid legitimate exam sharing.

CSV-to-database mapping

Retained input	Database destination
users.csv	university_user, role, user_role and organization_unit
spaces.csv	resource; preserve IDs, facilities and source order
requests.csv	resource_request, request_slot and exam_details
request_participants.csv	request_participant; preserve participant order
user_busy_slots.csv	busy_slot; preserve weekly intervals and reasons
config.txt	application_state strategy preference

Import supports dry-run, validates references, preserves IDs and ordering, advances identity sequences and commits all inputs together. Populated targets are refused; source files are never rewritten. Generated old results are not trusted migration inputs: the real engine regenerates decisions. Standalone C++ CSV CLI and explicit import remain supported.

Schema version: Alembic head `0041e`. Revisions 0041/0041a establish storage and participant ordering; 0041b adds integrity/index improvements; 0041c removes a redundant index; 0041d enforces deferred request subtype completeness; 0041e adds keyed form retry metadata. Invalid old data aborts upgrades without silent deletion.

9 Iteration Comparison

Concern	Before database migration	Iteration 41
Operational persistence	CSV/config and a local published result document	PostgreSQL records and transactional publication
Engine integration	Versioned JSON execution already available	Same JSON protocol; database-built snapshots
Result views	File-based result access / compatibility interfaces	One pinned database view; file-free presentation

10 Testing and Verification

The recorded Iteration 41 verification covers real PostgreSQL, not a substitute database. Latest merged-main Linux CI completed successfully on 13 October 2026.

Recorded verification	Result
Latest Linux C++ suite	140 passed; 0 failed
Latest Linux Python / PostgreSQL suite	156 passed; 0 failed; 0 skipped
Earlier full Windows verification	140 C++ passed; Python 154 passed, 0 failed, 1 POSIX-only skip
Strategy parity	All five strategies match equivalent original inputs, including allocations and suggestions
Migration / recovery	Fresh upgrades, drift, imports, identity continuation, rollback, pg_dump/pg_restore and restart checked

Regression coverage includes concurrent thread/process saves, lock timeout, failed/stale publication, coherent readers, subtype write-skew protection, hierarchy cycles and keyed submission retries. Flask workflows, schedules and restart are checked with runtime CSV/config/local-result access forbidden. Large snapshot construction uses bounded bulk queries; the tested limit is 15 SQL statements, not a production scalability guarantee.

Evidence: merged-main CI run 38365817054. For this report, the ORM table count, generated diagram/catalog consistency and Alembic head were checked; historical full-suite results above are not presented as a new local full-suite run.

11 Demonstration Data and Recovery

Fictional input	Default dataset	Large dataset
Users	106	1,060
Resources	20	200
Requests	120	1,200
Busy slots	12	120

A configurable seed reproduces fictional users, faculties/departments, room facilities and all four request types, including conflicts. Names are fictional and email addresses use example.invalid. The seed tool refuses populated targets and inserts no arbitrary approved allocations; `--run-allocation` invokes the real engine.

```
python scripts/seed_demo_database.py --seed 41 --dry-run
python scripts/seed_demo_database.py --seed 41 --run-allocation
```

Run only after migrating a new empty development database. Backup/restore verification uses matching PostgreSQL client tools and a separate empty restore target; original databases and regression fixtures are not overwritten.

12 Remaining Limitations

Scheduling remains weekly, without calendar dates or time zones. Authentication and student-professor appointments are not implemented. Run/history retention, abandoned-run reconciliation and engine-job deduplication remain administrative follow-up work; keyed form retries do not provide authentication or CSRF protection. Whole-batch rendering remains appropriate only for the bounded prototype.

13 Summary

Iteration 41 establishes a coherent PostgreSQL foundation while preserving tested C++ allocation behavior and JSON integration. Safe imports, structural constraints, short transactions, atomic publication and recovery checks improve integrity and maintainability. The result is a verified persistence baseline, not a claim of production readiness.